

CSS 보조자료

display 속성 완전 정복

block · inline · flex · grid

CSS를 처음 배우는 사람을 위한 배치의 원리

목차

1장. display 속성은 왜 필요한가

- 1-1 같은 코드인데 왜 다르게 보일까
- 1-2 모든 태그에는 타고난 성질이 있다
- 1-3 display가 하는 일

2장. 기본 4형제 — block · inline · inline-block · none

- 2-1 block
- 2-2 inline
- 2-3 inline이 겪는 문제
- 2-4 inline-block
- 2-5 none
- 2-6 한눈에 비교

3장. 배치 방법의 역사 — 왜 flex와 grid가 등장했나

4장. display: flex 완전 정복

- 4-1 부모와 자식
- 4-2 가장 중요한 개념 : 주축과 교차축
- 4-3 flex-direction
- 4-4 justify-content
- 4-5 align-items
- 4-6 flex-wrap과 gap
- 4-7 자식에게 주는 속성 (flex: 1)
- 4-8 실전 패턴 6가지
- 4-9 자주 하는 실수

5장. display: grid 완전 정복

- 5-1 grid는 무엇이 다른가
- 5-2 열과 행 정의하기
- 5-3 fr 단위
- 5-4 gap
- 5-5 grid-template-areas
- 5-6 반응형 자동 배치
- 5-7 자주 하는 실수

6장. flex와 grid, 무엇을 쓸까

7장. 함께 알아야 할 속성

7-1 box-sizing: border-box

7-2 overflow

7-3 position (참고)

부록. 자주 쓰는 레시피 · 실수 점검표

1 장. display 속성은 왜 필요한가

CSS를 배우다 보면 `display: flex`, `display: grid` 같은 코드를 만나게 됩니다. 그런데 그 전에 답해야 할 질문이 있습니다. "display가 대체 무엇을 정하는 속성인가?" 이것을 이해하지 않고 flex와 grid를 외우면, 왜 어떤 때는 되고 어떤 때는 안 되는지 영원히 알 수 없습니다.

1-1. 같은 코드인데 왜 다르게 보일까

아래 두 코드를 비교해 보세요. 태그 이름만 다르고 내용은 완전히 같습니다.

▶ 코드 A

```
<div>사과</div>
<div>바나나</div>
<div>포도</div>
```

▶ 코드 B

```
<span>사과</span>
<span>바나나</span>
<span>포도</span>
```

CSS는 한 줄도 쓰지 않았는데 화면에 나타나는 모습은 완전히 다릅니다.

코드 A (div)	코드 B (span)
사과 바나나 포도	사과 바나나 포도

div는 한 줄씩 차지하며 세로로 쌓이고, span은 옆으로 나란히 붙습니다. 왜 그럴까요? CSS를 안 썼는데도 다르다는 것은, 태그마다 이미 정해진 성질이 있다는 뜻입니다.

1-2. 모든 태그에는 타고난 성질이 있다

HTML은 원래 "문서"를 표현하기 위해 만들어졌습니다. 논문이나 책을 떠올려 보세요. 문서에는 두 종류의 요소가 있습니다.

종류	문서에서의 역할	해당하는 태그
덩어리(block)	문단, 제목, 표처럼 한 줄을 통째로 차지하는 것	div, p, h1~h6, ul, li, section, header, footer, form, table
글자(inline)	문장 안에서 일부 글자만 강조하거나 링크를 거는 것	span, a, strong, em, img, input, button, label

책에서 문단은 항상 새 줄에서 시작하고, 문단 안에서 굵게 표시한 단어는 문장 흐름을 끊지 않습니다.

HTML 태그의 기본 성질은 이 문서의 논리를 그대로 따라간 것입니다.

div가 세로로 쌓이는 것은 CSS 때문이 아니라, div가 원래 그런 성질을 갖고 태어났기 때문입니다.

1-3. display가 하는 일

메뉴는 가로로 놓고 싶고, 카드는 3개씩 나란히 배치하고 싶습니다. 태그가 타고난 성질만으로는 부족합니다. 바로 이때 쓰는 것이 display 속성입니다.

**display = "이 요소가 어떤 성질로 동작할지"와
"자식들을 어떤 규칙으로 배치할지"를 정하는 속성**

display는 두 가지 일을 동시에 합니다. 이 구분이 매우 중요합니다.

역할	설명
① 자기 자신의 성질	나는 한 줄을 차지할 것인가(block), 글자처럼 옆에 붙을 것인가(inline)
② 자식들의 배치 규칙	내 안에 든 자식들을 어떻게 늘어놓을 것인가 (flex, grid)

display: block이나 display: inline은 ①만 바꿉니다. 반면 display: flex나 display: grid는 ②까지 바꿉니다. 그래서 flex와 grid는 반드시 부모에게 선언해야 하는 것입니다.

■ **알아두기** "왜 flex를 자식이 아니라 부모에게 쓰나요?" — 가장 많이 나오는 질문입니다. 답은 이것입니다. flex는 "나를 어떻게 보이게 하라"는 명령이 아니라 "내 자식들을 이렇게 줄 세워라"는 명령이기 때문입니다. 반장이 반 아이들을 줄 세우는 것이지, 아이들이 스스로 줄을 서는 것이 아닙니다.

display 값의 전체 지도

```
display
|-- 자기 자신의 성질을 바꾸는 값
|   |-- block          한 줄을 통째로 차지
|   |-- inline        글자처럼 옆에 붙음
|   |-- inline-block   옆에 붙되 크기는 지정 가능
|   |-- none          화면에서 완전히 사라짐
|-- 자식들의 배치 규칙을 바꾸는 값
|   |-- flex           한 방향으로 줄 세우기 (1차원)
|   |-- grid           바둑판으로 배치 (2차원)
```

2장에서 위쪽 네 가지를, 4~5장에서 아래쪽 두 가지를 다룹니다. 순서를 건너뛰지 마세요. 위쪽을 모르면 아래쪽에서 반드시 막힙니다.

2 장. 기본 4 형제 — block · inline · inline-block · none

2-1 display: block

가로 한 줄을 통째로 차지합니다. 내용이 짧아도 남은 공간을 자기 영역으로 가져갑니다.

```
.box { display: block; background: pink; }
```

결과 (배경색을 주면 영역이 눈에 보입니다)

```

+-----+
| 사과                                     | <- 짧아도 한 줄 전체
+-----+
+-----+
| 바나나                                 |
+-----+
```

- 앞뒤로 줄바꿈이 일어납니다. 다음 요소는 반드시 아래로 내려갑니다.
- width, height 를 자유롭게 지정할 수 있습니다.
- margin, padding 이 상하좌우 모두 적용됩니다.
- 기본 태그 : div, p, h1~h6, ul, ol, li, section, article, header, footer, form, table

2-2 display: inline

글자처럼 취급됩니다. 내용의 크기만큼만 차지하고, 옆 요소와 나란히 이어집니다.

```
.text { display: inline; background: yellow; }
```

결과

```

+---+ +---+ +---+
|사과| |바나나| |포도| <- 내용 크기만큼만, 옆으로 이어짐
+---+ +---+ +---+
```

- 줄바꿈이 일어나지 않습니다. 한 줄이 꽉 차면 그때 다음 줄로 넘어갑니다(글자와 같은 방식).
- 기본 태그 : span, a, strong, em, img, input, button, label

2-3 inline이 겪는 문제 (반드시 알아야 할 함정)

inline 요소에는 심각한 제약이 있습니다. 초보자가 "분명히 CSS를 썼는데 아무 반응이 없다"고 느끼는 상황의 대부분이 여기서 나옵니다.

```

span {
  width: 200px;      /* 적용 안 됨! */
  height: 100px;     /* 적용 안 됨! */
}
```

```
margin-top: 50px; /* 적용 안 됨! */
padding-top: 50px; /* 겹보기만 커지고 다른 요소를 밀어내지 못함 */
}
```

속성	inline에서	이유
width / height	무시됨	글자는 폰트 크기가 크기를 결정하므로
margin 위/아래	무시됨	줄 간격은 line-height가 담당하므로
margin 좌/우	적용됨	글자 사이 간격은 별릴 수 있으므로
padding 위/아래	겹보기만 적용	배경은 커지지만 다른 요소를 밀어내지 못해 위아래 요소와 겹쳐 보임

■ **자주 나는 오류** 버튼처럼 보이게 하려고 <a> 태그에 width와 padding을 줬는데 크기가 안 변한다.
→ a는 inline입니다. display: inline-block 또는 display: block을 추가하세요.
span에 height를 줬는데 무시된다. → 같은 이유입니다.

2-4 display: inline-block

두 성질의 장점만 합친 값입니다. 옆으로 나란히 놓이면서(inline), 크기와 여백을 마음대로 지정할 수 있습니다(block).

```
.btn {
  display: inline-block;
  width: 120px;
  height: 40px;
  padding: 10px;
}
```

결과

```
+-----+ +-----+ +-----+
|         | |         | |         |
| 버튼1  | | 버튼2  | | 버튼3  |
+-----+ +-----+ +-----+
```

<- 옆으로 나란히,
크기 지정도 가능

■ **알아두기** Flexbox가 나오기 전에는 요소를 가로로 나열할 때 inline-block을 많이 썼습니다. 다만 HTML 코드 사이의 줄바꿈이 공백 문자로 해석되어, 요소 사이에 원하지 않는 4~5px 틈이 생기는 고질적인 문제가 있었습니다. 지금은 이런 용도라면 flex를 쓰는 것이 훨씬 낫습니다.

2-5 display: none

화면에서 완전히 사라집니다. 자리도 남기지 않습니다.

```
.hidden { display: none; }
```

비슷해 보이지만 전혀 다른 속성이 있습니다. 둘의 차이는 면접에서도 자주 나옵니다.

구분	display: none	visibility: hidden
보이는가	안 보임	안 보임
자리를 차지하는가	차지하지 않음 (없는 것과 같음)	자리는 그대로 남음 (빈 공간)
쓰는 경우	탭 전환, 모달 창 열고 닫기	레이아웃을 유지한 채 잠시 감출 때

display: none 일 때

visibility: hidden 일 때

[A]

[C]

[A]

[]

[C]

<- B의 자리는 남아 있음

실습에서 취소 버튼을 `style="display: none;"`으로 숨겼다가 JavaScript로 `display = "inline-block"`으로 바꿔 보이게 하는 것이 바로 이 원리입니다.

2-6 한눈에 비교

	block	inline	inline-block	none
줄바꿈	일어남	안 일어남	안 일어남	해당 없음
차지하는 폭	한 줄 전체	내용만큼	내용 또는 지정값	없음
width / height	적용	무시	적용	해당 없음
margin 상하	적용	무시	적용	해당 없음
대표 태그	div, p, h1	span, a, img	(직접 지정)	(직접 지정)

■ **미니 과제** <a> 태그 3개를 만들고 배경색과 padding을 줘보세요. 그다음 display: inline-block을 추가해서 무엇이 달라지는지 확인하세요.

3 장. 배치 방법의 역사 — 왜 flex 와 grid 가 등장했나

block과 inline만으로는 요즘 웹사이트를 만들 수 없습니다. "메뉴를 가로로 놓고 오른쪽 끝에 붙이기", "카드를 3개씩 나란히", "글자를 상자 정중앙에" 같은 요구를 해결할 수 없기 때문입니다. 그래서 웹 개발자들은 오랫동안 편법을 써왔고, 그 결과 flex와 grid가 만들어졌습니다.

시대	방법	문제점	지금은
~2015년	float, position 을 이용한 편법	계산이 복잡하고 버그가 잦음 반응형이 어려움	표 형태 레이아웃에는 거의 쓰지 않음
2015년~	Flexbox (1차원 배치)	가로 또는 세로 한 방향만 제어	메뉴, 버튼, 카드 내부 배치의 표준
2017년~	Grid (2차원 배치)	(구형 브라우저)	페이지 전체 레이아웃의 표준

float 시절의 코드

```
/* 옛날 방식 - 참고만 하세요 */
.item {
  float: left;
  width: 33.333%;      /* gap을 넣으면 이 숫자를 다시 계산해야 함 */
  margin-right: 10px;  /* 마지막 요소만 빼야 하는데 방법이 복잡함 */
}
.container::after {
  content: "";
  display: block;
  clear: both;         /* 이 주문을 외우지 않으면 레이아웃이 무너짐 */
}
```

float은 원래 "이미지 옆에 글이 흐르게 하는" 용도로 만들어진 속성입니다. 배치 도구가 없어서 이 속성을 억지로 끌어다 쓴 것입니다. clear: both 같은 주문을 외워야 했고, 세로 중앙 정렬은 사실상 불가능했습니다.

flex와 grid는 "배치를 위해 처음부터 새로 설계된" 최초의 도구입니다.

그래서 하고 싶은 말을 그대로 코드로 쓸 수 있습니다.

예를 들어 "가운데로 모아라"는 justify-content: center 한 줄이고, "3등분해라"는 repeat(3, 1fr) 한 줄입니다. 계산이 필요 없습니다.

4 장. display: flex 완전 정복

Flexbox는 "한 방향으로 줄을 세우는" 도구입니다. 이름 그대로 유연한(flexible) 상자라는 뜻이며, 남은 공간을 알아서 나눠 갖고 부족하면 알아서 줄어듭니다.

4-1. 부모와 자식 — 누구에게 무엇을 쓰는가

Flexbox를 이해하려면 딱 하나만 기억하면 됩니다. 등장인물이 둘이라는 것입니다.

```
<div class="container">      <-- 부모 : Flex 컨테이너 (여기에 display: flex)
  <div class="item">1</div>    <-- 자식 : Flex 아이탬
  <div class="item">2</div>    <-- 자식
  <div class="item">3</div>    <-- 자식
</div>
```

구분	누구에게 쓰나	어떤 속성
배치 규칙	부모 (컨테이너)	display, flex-direction, justify-content, align-items, flex-wrap, gap
개별 조정	자식 (아이템)	flex-grow, flex-shrink, flex-basis, flex, align-self, order

■ **알아두기** 속성이 안 먹는다면 90%는 "부모에게 써야 할 것을 자식에게 썼거나 그 반대"입니다. justify-content를 자식에게 쓰면 아무 일도 일어나지 않습니다.

4-2. 가장 중요한 개념 : 주축과 교차축

Flexbox에는 두 개의 축이 있습니다.

1단계 – 축을 결정하는 속성

속성	값	주축	교차축
flex-direction	row(기본값)	가로 →	세로 ↓
	column	세로 ↓	가로 →

2단계 — 결정된 축 위에서 동작하는 속성

축	이름	담당 속성
주축 (main axis)	아이템이 나열되는 방향	justify-content
교차축 (cross axis)	주축과 직각인 방향	align-items

`flex-direction: row` (기본값)

```

      주축 (가로) ----->
      +-----+ +-----+ +-----+
교차 |   1   | |   2   | |   3   |
축   +-----+ +-----+ +-----+
|
|
v

```

`flex-direction: column`

```

      교차축 (가로) ----->
      +-----+
주   |   1   |
축   +-----+
|   +-----+
|   |   2   |
|   +-----+
v

```

`justify-content` = 주축 정렬, `align-items` = 교차축 정렬
"가로/세로"가 아니라 "주축/교차축"으로 외워야 합니다.

많은 입문 자료가 "justify-content는 가로 정렬"이라고 설명하는데, 이것은 flex-direction이 row일 때만 맞습니다. column으로 바꾸는 순간 justify-content가 세로를 담당하게 되어 학습자가 크게 혼란스러워합니다. 처음부터 주축/교차축으로 익히는 편이 결국 더 빠릅니다.

4-3. flex-direction — 줄 세우는 방향

```

.container {
  display: flex;
  flex-direction: row; /* 기본값이라 생략 가능 */
}

```

값	방향	결과
row (기본값)	왼쪽 → 오른쪽	[1] [2] [3]
row-reverse	오른쪽 → 왼쪽	[3] [2] [1]
column	위 → 아래	[1] [2] [3]
column-reverse	아래 → 위	[3] [2] [1]

display: flex만 쓰고 방향을 지정하지 않으면 자동으로 row가 됩니다. 그래서 div들이 갑자기 가로로 나란히 서는 것입니다.

4-4. justify-content — 주축 정렬 (6가지 값)

주축 방향으로 아이টে를 어디에 모을지 정합니다. 아래 그림에서 점(.)은 빈 공간입니다.

flex-start (기본값)	[1][2][3].....
flex-end	[1][2][3]
center	[1][2][3].....
space-between	[1].....[2].....[3] (양 끝에 딱 붙이고 사이를 최대로)
space-around	[1].....[2].....[3].... (각 아이টে 좌우에 같은 여백)
space-evenly	[1].....[2].....[3]..... (모든 틈이 완전히 같음)

값	언제 쓰나
flex-start	기본값. 특별히 정렬할 필요가 없을 때
center	버튼 안의 글자, 로고 하나만 있는 상단 바
flex-end	오른쪽 하단의 확인/취소 버튼 묶음
space-between	로고는 왼쪽, 메뉴는 오른쪽인 네비게이션 바 (가장 많이 씀)
space-around	아이콘 메뉴를 균등하게 흩뿌릴 때
space-evenly	간격을 완전히 똑같이 하고 싶을 때

■ **알아두기** space-between과 space-around의 차이가 잘 안 보인다면, 양쪽 끝을 보세요. space-between은 첫 아이টে와 마지막 아이টে가 벽에 딱 붙습니다. space-around는 벽과 아이টে 사이에도 여백이 있습니다.

4-5. align-items — 교차축 정렬 (5가지 값)

flex-direction이 row일 때는 세로 정렬을 담당합니다. 부모의 높이가 아이টে보다 클 때 의미가 생깁니다.

부모 높이 100px, 아이টে 높이 30px 일 때

flex-start	<pre> +-----+ [1] [2] [3] +-----+ </pre>
------------	--

center	<pre> +-----+ [1] [2] [3] +-----+ </pre>	
flex-end	<pre> +-----+ [1] [2] [3] +-----+ </pre>	
stretch (기본값)	<pre> +-----+ [1] [2] [3] [1] [2] [3] +-----+ </pre>	<- 아이템이 세로로 늘어나 부모 높이를 꽉 채움

값	설명
stretch (기본값)	아이템이 교차축 방향으로 늘어나 부모를 꽉 채움. 아이템에 height가 지정되어 있으면 적용되지 않음
flex-start	교차축 시작점(위쪽)에 붙임
center	교차축 정중앙. 가장 많이 사용
flex-end	교차축 끝(아래쪽)에 붙임
baseline	글자의 아랫선을 기준으로 맞춤. 글자 크기가 다른 요소를 나란히 놓을 때 유용

■ 자주 나는 오류 align-items: center를 썼는데 아무 변화가 없다. → 부모의 높이가 아이템 높이와 같아서 움직일 공간이 없는 경우입니다. 부모에 height를 주거나 padding을 주면 효과가 보입니다.

가장 많이 쓰는 조합 : 완전 중앙 정렬

```

.center-box {
  display: flex;
  justify-content: center; /* 주축(가로) 중앙 */
  align-items: center;     /* 교차축(세로) 중앙 */
  height: 300px;          /* 높이가 있어야 세로 중앙이 의미 있음 */
}

```

과거에는 세로 중앙 정렬을 위해 요소 높이를 계산해서 margin-top에 음수를 넣거나, 테이블 태그를 흉내 내는 편법을 썼습니다. 지금은 이 세 줄이면 끝납니다. Flexbox의 가장 큰 선물입니다.

4-6. flex-wrap과 gap

flex-wrap : 줄바꿈

Flex 아이템은 기본적으로 줄바꿈을 하지 않습니다. 공간이 부족하면 아이템을 억지로 찌그러뜨립니다.

```
flex-wrap: nowrap (기본값)
[1][2][3][4][5][6]      <- 넘치면 각 아이템이 찢그러짐

flex-wrap: wrap
[ 1 ][ 2 ][ 3 ]
[ 4 ][ 5 ][ 6 ]          <- 공간이 부족하면 다음 줄로 내려감
```

태그 목록이나 카드 목록처럼 개수가 정해지지 않은 요소를 나열할 때는 거의 항상 flex-wrap: wrap을 함께 씁니다.

gap : 아이템 사이 간격

```
.container {
  display: flex;
  gap: 10px;      /* 아이템 사이만 10px */
  gap: 10px 20px; /* 행 간격 10px, 열 간격 20px */
}
```

예전에는 margin-right: 10px을 주고 마지막 아이템만 :last-child로 제거하는 번거로운 작업을 했습니다. gap은 "사이"에만 간격을 넣기 때문에 양쪽 끝에 불필요한 여백이 생기지 않습니다.

4-7. 자식에게 주는 속성 — flex: 1의 정체

여기까지는 모두 부모에게 쓰는 속성이었습니다. 이제 자식 하나하나를 조정하는 속성을 봅니다.

```
.item {
  flex-grow: 1;      /* 남는 공간이 있으면 얼마나 차지할지 (기본 0) */
  flex-shrink: 1;    /* 공간이 부족할 때 얼마나 줄어들지 (기본 1) */
  flex-basis: auto;  /* 기본 크기 (기본 auto) */

  flex: 1;           /* 위 세 개를 한 번에 = 1 1 0 */
}
```

flex-grow 이해하기

```
부모 너비 600px, 아이템 3개

아무 설정 없음 (flex-grow: 0)
[1][2][3].....      <- 내용 크기만큼만, 나머지는 빈 공간

모두 flex: 1
[ 1 ][ 2 ][ 3 ]      <- 남는 공간을 1:1:1로 나눠 가짐

2번만 flex: 1
[1][      2      ][3]      <- 남는 공간을 2번이 독차지

1번 flex:1, 2번 flex:2, 3번 flex:1
[ 1 ][      2      ][ 3 ]      <- 1:2:1 비율로 나눔
```

■ **알아두기** flex: 1은 "남는 공간을 균등하게 나눠 가져라"라는 뜻입니다. 사이드바는 고정 폭, 본문은 나머지 전부인 레이아웃을 만들 때 가장 많이 씁니다.

```
.sidebar { width: 200px; } /* 고정 */
.content { flex: 1; } /* 나머지 전부 */
```

그 밖의 자식 속성

속성	설명
align-self	이 아이템만 다른 교차축 정렬을 적용. 예: 나머지는 위쪽, 나만 아래쪽
order	숫자가 작을수록 앞. HTML 순서를 바꾸지 않고 화면 순서만 바꿀 때 사용 (기본 0)

4-8. 실전 패턴 6가지 (이것만 외워도 충분합니다)

① 네비게이션 바 — 로고 왼쪽, 메뉴 오른쪽

```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 0 40px;
  height: 70px;
}
```

결과 : [로고]

[메뉴1 메뉴2 메뉴3]

② 완전 중앙 정렬

```
.hero {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 400px;
}
```

③ 아이콘 + 텍스트 나란히 (세로 중앙 맞춤)

```
.label {
  display: flex;
  align-items: center; /* 아이콘과 글자의 세로 중심을 맞춤 */
  gap: 8px;
}
```

결과 : [아이콘] 알람 3건

④ 사이드바 + 본문

```
.layout { display: flex; }
.sidebar { width: 250px; } /* 고정 */
.content { flex: 1; } /* 나머지 전부 */
```

⑤ 카드 목록 (줄바꿈 포함)

```
.card-list {
  display: flex;
  flex-wrap: wrap;
}
```

```
gap: 20px;
}
.card { width: 300px; }
```

⑥ 세로 배치 — 라벨 위, 입력창 아래

```
.form-group {
  display: flex;
  flex-direction: column;
  gap: 5px;
}
```

입력 폼을 만들 때 자주 쓰는 패턴입니다. flex-direction: column을 쓰면 이 안에서는 justify-content가 세로, align-items가 가로를 담당하게 된다는 점을 다시 확인해 보세요.

4-9. 자주 하는 실수 5가지

증상	원인	해결
가로로 안 늘어선다	자식에게 display: flex를 씀	부모에게 선언
justify-content가 안 먹는다	부모에 display: flex가 없음	display: flex 먼저 선언
align-items: center가 반응이 없다	부모에 높이가 없어 움직일 공간이 없음	부모에 height 또는 padding 지정
column인데 justify-content가 세로로 동작해서 당황	정상 동작임	주축/교차축 개념 다시 확인 (4-2)
아이템이 찌그러진다	기본값이 nowrap이라 역지로 줄어들	flex-wrap: wrap 추가 또는 flex-shrink: 0

■ **미니 과제** div 5개를 만들고 부모에 display: flex를 준 뒤, justify-content의 6가지 값을 하나씩 바꿔 보며 화면을 관찰하세요.

그다음 flex-direction: column으로 바꾸고 같은 값들을 다시 적용해 보세요. 방향이 어떻게 바뀌나요?

Interactive CSS Flexbox Playground (주축/교차축 개념 확인용으로 가장 직관적임)

<https://dylanbeattie.net/tools/flexbox/>

Flexbox Playground (컨테이너와 개별 아이템 속성을 따로 조절할 수 있습니다.)

<https://tools.simonwillison.net/flexbox-playground>

Flexbox Foggy (CSS를 직접 입력해 개구리를 연잎으로 보내는 24단계 게임)

<https://flexboxfroggy.com/#ko>

5 장. display: grid 완전 정복

5-1. grid는 무엇이 다른가

Flexbox가 "한 줄로 세우기"라면, Grid는 "모눈종이에 칸을 나누고 그 안에 물건을 놓기"입니다. 가로(행)와 세로(열)를 동시에 제어하는 2차원 시스템입니다.

Flexbox (1차원)	Grid (2차원)
[1][2][3][4] 한 방향으로만 통제	<pre> +---+---+---+ 1 2 3 +---+---+---+ 4 5 6 +---+---+---+ </pre> 행과 열을 동시에 통제

Flex는 아이템의 크기가 배치를 결정하지만(콘텐츠 기준), Grid는 먼저 칸을 그려놓고 그 안에 아이템을 넣습니다(레이아웃 기준). 이 차이가 6장의 선택 기준으로 이어집니다.

5-2. 열과 행 정의하기

```

.container {
  display: grid;
  grid-template-columns: 100px 200px 100px; /* 3열 */
  grid-template-rows: 50px 50px; /* 2행 */
  gap: 10px;
}

```

```

<div class="container">
  <div>1</div> <div>2</div> <div>3</div>
  <div>4</div>
</div>

```

결과

1	2	3	<- 높이 50px
4			<- 높이 50px
100px	200px	100px	

자식 요소에는 아무 CSS도 쓰지 않았습니다. 부모가 칸을 그려놓으면 자식들이 왼쪽 위부터 순서대로 채워 들어갑니다. 이것이 Grid의 기본 동작입니다.

■ **알아두기** grid-template-rows를 생략해도 됩니다. 그러면 행의 개수와 높이는 내용에 맞춰 자동으로 정해집니다. 실무에서는 열만 정의하고 행은 자동으로 맡기는 경우가 훨씬 많습니다.

5-3. fr 단위 — Grid의 핵심

fr은 fraction(비율)의 줄임말로, 사용할 수 있는 공간을 나누는 단위입니다. px처럼 절대적인 크기가 아니라 "남은 공간의 몇 등분"을 뜻합니다.

```
grid-template-columns: 1fr 1fr 1fr;
+-----+-----+-----+      전체를 정확히 3등분

grid-template-columns: 2fr 1fr;
+-----+-----+      왼쪽이 오른쪽의 2배

grid-template-columns: 200px 1fr 100px;
+-----+-----+-----+      고정 - 유연 - 고정
200px      남은 공간 전부 100px

grid-template-columns: repeat(4, 1fr);
= 1fr 1fr 1fr 1fr 과 완전히 같음 (반복이 많을 때 편리)
```

px은 "몇 픽셀", fr은 "남은 공간의 몇 등분".

브라우저 창 크기가 바뀌어도 fr은 알아서 다시 계산됩니다.

200px 1fr 100px 같은 조합이 특히 유용합니다. 사이드바와 광고 영역은 고정 폭으로 두고, 본문만 화면 크기에 따라 늘어나게 만들 수 있습니다.

5-4. gap — 칸 사이의 간격

```
gap: 10px;           /* 행,열 모두 10px */
gap: 20px 10px;      /* 행 간격 20px, 열 간격 10px */
row-gap: 20px;       /* 행만 */
column-gap: 10px;    /* 열만 */
```

gap은 칸과 칸 "사이"에만 들어갑니다. 바깥 테두리에는 여백이 생기지 않으므로, 예전처럼 마지막 요소의 margin을 따로 제거할 필요가 없습니다.

5-5. grid-template-areas — 그림 그리듯 배치하기

Grid의 가장 독특하고 강력한 기능입니다. 레이아웃을 글자로 그려서 그대로 코드에 옮길 수 있습니다.

```
.website {
  display: grid;
  grid-template-areas:
    "header header header"
    "sidebar content content"
    "sidebar content content"
    "footer footer footer";
}
```

```

grid-template-columns: 200px 1fr 1fr;
grid-template-rows: 80px 1fr 1fr 60px;
gap: 10px;
height: 400px;
}

```

```

.header { grid-area: header; }
.sidebar { grid-area: sidebar; }
.content { grid-area: content; }
.footer { grid-area: footer; }

```

결과

+-----+-----+-----+			
	HEADER (헤더)		80px
+-----+-----+-----+			
	SIDEBAR	CONTENT (본문)	1fr
	(메뉴)		1fr
+-----+-----+-----+			
	FOOTER (푸터)		60px
+-----+-----+-----+			
200px		1fr + 1fr	

규칙 세 가지

- ① 따옴표로 묶인 한 줄이 한 행(row)입니다. 위 예시는 4줄이므로 4행입니다.
- ② 같은 이름을 여러 칸에 쓰면 그 칸들이 하나로 합쳐집니다. "header header header"는 헤더가 3칸을 모두 차지한다는 뜻입니다.
- ③ 자식 요소에 grid-area: 이름 을 써서 어느 영역에 들어갈지 연결합니다.

■ **알아두기** 빈 칸을 만들고 싶으면 마침표를 씁니다. 예를 들어 "header header ." 라고 쓰면 오른쪽 한 칸은 비워둡니다. 또한 같은 이름은 반드시 직사각형 모양으로 이어져야 하며, 꺾거나 계단 모양은 만들 수 없습니다.

이 방식의 진짜 장점은 반응형입니다. 모바일에서는 areas만 다시 그리면 됩니다.

```

@media (max-width: 768px) {
  .website {
    grid-template-areas:
      "header"
      "content"
      "sidebar"
      "footer";
    grid-template-columns: 1fr;
  }
}

```

HTML은 한 글자도 고치지 않았는데 사이드바가 본문 아래로 내려갔습니다. HTML의 순서와 화면의 순서를 분리할 수 있다는 것이 Grid의 큰 강점입니다.

5-6. 반응형 자동 배치 — auto-fit과 minmax

카드 개수가 정해져 있지 않을 때, 화면 크기에 맞춰 열 개수를 알아서 조절하는 방법입니다. 미디어 쿼리 없이 반응형이 됩니다.

```
.card-list {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 20px;
}
```

부분	의미
auto-fit	몇 개의 열을 만들지 브라우저가 알아서 결정
minmax(250px, 1fr)	각 열은 최소 250px, 최대는 남는 공간을 나눠 가짐
합친 의미	"한 칸이 250px보다 작아지면 열 개수를 줄여라"
넓은 화면 (1200px)	[카드][카드][카드][카드]
중간 화면 (800px)	[카드][카드][카드]
좁은 화면 (500px)	[카드][카드]
모바일 (350px)	[카드]

■ **알아두기** 이 한 줄이 미디어 쿼리 여러 개를 대신합니다. 카드 목록, 상품 목록, 갤러리를 만들 때 가장 먼저 시도해 볼 코드입니다.

5-7. 자주 하는 실수

증상	원인	해결
칸이 안 나뉜다	자식에게 display: grid를 씀	부모에게 선언
grid-template-columns가 무시된다	display: grid가 없음	먼저 display: grid 선언
아이템이 예상과 다른 칸에 들어간다	자식 개수가 칸 수와 맞지 않음	개수를 확인하거나 자동 배치에 맡김
areas가 동작하지 않는다	이름이 직사각형이 아니거나 각 줄의 칸 수가 다름	모든 줄의 칸 수를 같게 맞춤
간격이 이상하다	margin과 gap을 함께 씀	Grid에서는 gap만 사용

Grid Garden (flex 다음 단계인 CSS Grid를 익히는 게임)

<https://cssgridgarden.com/#ko>

6 장. flex 와 grid, 무엇을 쓸까

둘 다 배치 도구라 헛갈립니다. 하지만 판단 기준은 의외로 단순합니다.

한 줄로 늘어놓는가? → flex

행과 열을 모두 맞춰야 하는가? → grid

결정 흐름

배치할 것이 있다

+-+ 가로 또는 세로 한 방향으로만 늘어놓으면 되나?

```
|
+-- 예      --> flex
|              (메뉴, 버튼 묶음, 아이콘+글자, 카드 내부)
```

+-- 아니오

+-----+-- 칸을 미리 나누고 그 안에 넣고 싶다

```
+-- 예 --> grid
            (페이지 전체 구조, 카드 목록, 갤러리)
```

+- 줄바꿈만 필요하다 --> flex + flex-wrap

기준	Flexbox	Grid
차원	1차원 (한 방향)	2차원 (행 + 열)
기준	콘텐츠 기준 (내용 크기가 배치를 결정)	레이아웃 기준 (칸을 먼저 그리고 내용을 넣음)
적합한 규모	작은 단위 버튼, 메뉴, 카드 안쪽	큰 단위 페이지 전체, 목록 전체
줄 맞춤	줄마다 아이템 수가 달라지면 세로줄이 안 맞춤	항상 세로줄이 정확히 맞춤
대표 코드	justify-content align-items flex: 1	grid-template-columns gap grid-template-areas

■ **알아두기** 둘은 경쟁 관계가 아닙니다. 실무에서는 함께 씁니다. 페이지 전체 구조는 Grid로 잡고, 그 안의 헤더 내부 배치는 Flex로 하는 식입니다. Grid 아이템 안에 다시 display: flex를 쓰는 것은 매우 흔한 패턴입니다.

```
.page { display: grid; grid-template-columns: 200px 1fr; } /* 큰 구조 */
.header { display: flex; justify-content: space-between; } /* 그 안의 배치 */
```

7 장. 함께 알아야 할 속성

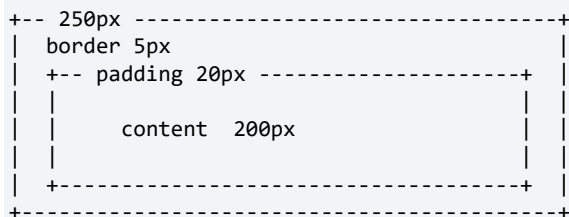
7-1. box-sizing: border-box

display와 함께 반드시 알아야 할 속성입니다. 이것을 모르면 "padding을 줬더니 레이아웃이 깨졌다"는 문제를 계속 만나게 됩니다.

기본값(content-box)의 문제

```
.box {
  width: 200px;
  padding: 20px;
  border: 5px solid red;
}
```

실제 차지하는 너비
 = 200(내용) + 20x2(패딩) + 5x2(테두리)
 = 250px <-- 200px로 정했는데 250px이 되어버림!



border-box 로 바꾸면

```
.box {
  width: 200px;
  padding: 20px;
  border: 5px solid red;
  box-sizing: border-box; /* 핵심 */
}
```

실제 차지하는 너비 = 200px (변하지 않음)
 내용 영역 = 200 - 40 - 10 = 150px (안쪽이 대신 줄어들음)

	content-box (기본값)	border-box
width의 의미	내용 영역만의 너비	테두리까지 포함한 전체 너비
padding을 주면	상자가 커짐	상자 크기는 그대로, 내용이 줄어들음
결과	레이아웃이 밀림	예측한 대로 배치됨

그래서 대부분의 프로젝트는 CSS 맨 위에 아래 세 줄을 넣고 시작합니다.

```
* {
  box-sizing: border-box;
}
```

■ **알아두기** Grid에서 1fr을 쓸 때 특히 중요합니다. 칸 크기를 브라우저가 계산해 주는데, padding 때

문에 실제 크기가 달라지면 3등분이 3등분이 아니게 됩니다.

7-2 overflow — 내용이 상자를 넘칠 때

내용이 상자보다 클 때 어떻게 처리할지 정합니다.

값	동작	주로 쓰는 곳
visible (기본값)	넘치는 내용을 그대로 보여줌 상자 밖으로 빠져나옴	기본 상태. 의도치 않으면 레이아웃이 깨져 보임
hidden	넘치는 부분을 잘라내어 감춤	둥근 모서리 안에 이미지 맞추기, 이미지 확대 효과 가두기
scroll	스크롤바를 항상 표시	내용이 적어도 스크롤바가 보여 잘 쓰지 않음
auto	넘칠 때만 스크롤바 표시	채팅창, 모달, 사이드바 등 가장 많이 사용

overflow: hidden 의 대표 용도

```
.card {
  border-radius: 15px;
  overflow: hidden; /* 없으면 이미지가 둥근 모서리를 덮어버림 */
}
```

카드 모서리를 둥글게 깎아도, 그 안에 넣은 직사각형 이미지는 모서리를 그대로 덮어버립니다.

overflow: hidden이 빠져나온 부분을 잘라내어 매끈한 둥근 모양을 유지해 줍니다.

■ 자주 나는 오류 overflow: hidden을 부모에 줬더니 드롭다운 메뉴나 툴팁이 잘려서 안 보인다. → 자식이 부모 밖으로 나가야 하는 요소라면 hidden을 쓰면 안 됩니다. 이 경우 부모의 overflow를 visible로 두고 다른 방법을 찾아야 합니다.

가로·세로를 따로 지정할 수도 있습니다.

```
overflow-x: auto; /* 가로만 스크롤 (넓은 표에 유용) */
overflow-y: hidden; /* 세로는 잘라냄 */
```

7-3 position (참고)

display가 "흐름 안에서 어떻게 배치할까"를 정한다면, position은 "흐름에서 벗어나 특정 위치에 고정할까"를 정합니다. 서로 다른 일을 하므로 함께 쓰입니다.

값	설명	쓰는 곳
static (기본값)	문서 흐름 그대로	일반적인 요소
relative	원래 자리를 유지한 채 조금 이동	자식의 absolute 기준점 역할
absolute	흐름에서 빠져나와 기준 요소에 붙음	벤티지, 닫기 버튼, 드롭다운
fixed	화면에 고정 (스크롤해도 유지)	상단 고정 메뉴, 채팅 버튼
sticky	스크롤하다 특정 위치에서 고정	표의 머리글, 목차

■ **알아두기** 레이아웃의 기본은 display(flex, grid)로 잡고, position은 "그 위에 겹쳐 놓아야 하는 것"에만 쓰세요. position만으로 전체 레이아웃을 만들려는 것이 3장에서 말한 옛날 방식의 어려움이었습니다.

부록 A. 복사해서 바로 쓰는 레시피

실무에서 반복해서 쓰게 되는 배치 코드 모음입니다. 처음에는 이해하려 애쓰기보다 그대로 써보고, 값을 하나씩 바꿔가며 익히세요.

1. 상단 네비게이션 바

```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  height: 70px;
  padding: 0 40px;
}
```

2. 무엇이든 정중앙에

```
.center {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 300px;
}
```

3. 반응형 카드 목록 (미디어 쿼리 없이)

```
.card-list {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 20px;
}
```

4. 사이드바 + 본문

```
.layout { display: grid; grid-template-columns: 250px 1fr; gap: 20px; }

/* 또는 flex로 */
.layout { display: flex; gap: 20px; }
.sidebar { width: 250px; }
.content { flex: 1; }
```

5. 태그·칩 나열 (개수 상관없이)

```
.tags {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
}
```

6. 라벨 위 · 입력창 아래

```
.form-group {
  display: flex;
  flex-direction: column;
  gap: 5px;
}
```

7. 푸터를 항상 화면 맨 아래로

```
body {
  display: flex;
  flex-direction: column;
  min-height: 100vh;
}
main { flex: 1; } /* 본문이 남은 공간을 모두 차지 */
```

내용이 짧은 페이지에서도 푸터가 화면 중간에 떠 있지 않고 맨 아래에 붙습니다. 자주 쓰이는 패턴입니다.

부록 B. 배치가 안 될 때 점검표

생각대로 배치되지 않을 때 위에서부터 순서대로 확인하세요. 대부분 3번 안에서 해결됩니다.

1. display를 자식이 아니라 부모에게 썼는가? (flex와 grid는 항상 부모)
2. 오타는 없는가? 클래스 이름, 콜론, 세미콜론, 하이픈(justify-content는 하이픈 하나)
3. F12 → Elements 탭에서 그 요소를 클릭했을 때, 오른쪽에 내가 쓴 CSS가 실제로 보이는가? 취소선이 그어져 있다면 다른 규칙에 밀린 것이다.
4. align-items가 안 먹는다면 부모에 높이가 있는가?
5. width가 안 먹는다면 그 요소가 inline은 아닌가? (a, span 등)
6. padding 때문에 크기가 커졌다면 box-sizing: border-box를 넣었는가?
7. 요소가 잘려 보인다면 부모에 overflow: hidden이 있지 않은가?

디버깅 요령 : 모든 상자에 테두리 그리기

배치가 왜 이상한지 모를 때, 아래 코드를 CSS 맨 아래에 잠시 넣어보세요. 모든 요소의 경계가 보여서 어디가 문제인지 즉시 드러납니다. 확인이 끝나면 지우면 됩니다.

```
* {
  outline: 1px solid red; /* border가 아니라 outline이라 크기에 영향 없음 */
}
```

■ **알아두기** border 대신 outline을 쓰는 이유는, border는 요소의 크기를 바꿔서 오히려 레이아웃을 더 헷갈리게 만들기 때문입니다. outline은 그래도 크기가 변하지 않습니다.

부록 C. 용어 정리

용어	뜻
컨테이너 (container)	display: flex 또는 grid를 선언한 부모 요소
아이템 (item)	컨테이너 안에 들어 있는 자식 요소들
주축 (main axis)	flex에서 아이템이 늘어서는 방향. justify-content가 담당
교차축 (cross axis)	주축과 직각인 방향. align-items가 담당
fr	fraction(비율). 남은 공간을 나누는 Grid 전용 단위
gap	아이템 사이의 간격. 바깥쪽에는 여백이 생기지 않음
vh / vw	화면 높이 / 너비의 1%. 100vh = 화면 높이 전체
rem	기본 글자 크기(보통 16px)를 1로 보는 상대 단위

마무리 — 이 자료에서 꼭 가져갈 다섯 문장

- ① 태그에는 타고난 성질(block / inline)이 있고, display는 그 성질과 자식 배치 규칙을 바꾸는 속성이다.
- ② inline 요소에는 width와 height가 적용되지 않는다. 크기를 주고 싶으면 inline-block이나 block으로 바꿔야 한다.
- ③ flex와 grid는 항상 부모에게 선언한다. 자식을 줄 세우는 명령이기 때문이다.
- ④ flex에서 justify-content는 주축, align-items는 교차축을 담당한다. flex-direction을 바꾸면 축도 함께 바뀐다.
- ⑤ 한 줄이면 flex, 행과 열을 모두 맞춰야 하면 grid. 둘은 함께 쓰는 것이 자연스럽다.